

GOVERNO DO ESTADO DO CEARÁ — SEDUC
E.E.E.P. Lúcia Helena Viana Ribeiro — Horizonte/CE

CONFIGURAÇÃO INICIAL DE UM PROJETO EXPRESS COM MVC

Node.js · ESM · Express 5 · Handlebars · Nodemon · ESLint

Roteiro guiado do aluno

Desenvolvimento Web — Eixo Informação e Comunicação
Prof. Francisco Ericson Cornélio da Costa

Aluno(a): _____ Turma: _____

Sumário

1. O que vamos construir
2. Pré-requisitos e verificação do ambiente
3. Etapa 1 — Criar o projeto e o package.json
4. Etapa 2 — Instalar as dependências
5. Etapa 3 — Ajustar o package.json (ESM e script dev)
6. Etapa 4 — Configurar o ESLint
7. Etapa 5 — Criar a estrutura de pastas src/
8. Etapa 6 — Criar o src/app.js
9. Etapa 7 — Criar o server.js
10. Etapa 8 — Configurar o Handlebars e os middlewares
11. Etapa 9 — Criar a primeira rota, controller e view
12. Etapa 10 — Executar e testar
13. Conferência final do projeto
14. Erros comuns e como resolver
15. Atividades

1. O que vamos construir

Ao final deste roteiro você terá uma aplicação web em Node.js rodando em `http://localhost:3000`, organizada no padrão MVC, com recarregamento automático a cada alteração de código.

Objetivos de aprendizagem

- Inicializar um projeto Node.js e entender o papel do `package.json`.
- Diferenciar dependências de produção (`dependencies`) e de desenvolvimento (`devDependencies`).
- Configurar o projeto para usar módulos ESM (`import / export`).
- Separar responsabilidades entre servidor, aplicação, rotas, controllers e views.
- Configurar o motor de templates Handlebars para renderizar páginas HTML.
- Diagnosticar os erros mais comuns de configuração inicial.

Estrutura final do projeto

Esta é a árvore de arquivos que você deve ter ao terminar. Use-a como referência durante todo o roteiro.

```

app_escola/
├── node_modules/
├── src/
│   ├── controllers/
│   │   └── homeController.js
│   ├── routes/
│   │   └── home.js
│   ├── views/
│   │   ├── layouts/
│   │   │   └── main.handlebars
│   │   └── home.handlebars
│   └── app.js
├── eslint.config.mjs
├── package-lock.json
├── package.json
└── server.js
  
```

O papel de cada peça

Arquivo / pasta	Responsabilidade
<code>server.js</code>	Único ponto que liga o servidor a uma porta. Não conhece rotas nem views.
<code>src/app.js</code>	Monta a aplicação Express: middlewares, motor de views e registro das rotas.
<code>src/routes/</code>	Define QUAIS endereços existem e qual método HTTP responde a cada um.
<code>src/controllers/</code>	Define O QUE acontece quando um endereço é acessado.
<code>src/views/</code>	Define COMO o resultado aparece para o usuário (HTML gerado pelo Handlebars).
<code>package.json</code>	Identidade do projeto: nome, dependências e scripts de execução.
<code>eslint.config.mjs</code>	Regras de qualidade e padronização do código.



DICA

Guarde esta frase: `server.js` liga, `app.js` monta, `routes` direciona, `controller` decide, `view` mostra.

2. Pré-requisitos e verificação do ambiente

Antes de começar, confirme que as ferramentas estão instaladas. Abra o terminal e execute os três comandos abaixo, um de cada vez.

Terminal

```
node -v  
npm -v  
code -v
```

✓ CHECKPOINT

node -v deve retornar uma versão igual ou superior a v18 (ex.: v22.x).

npm -v deve retornar um número de versão.

Se algum comando responder “command not found” ou “não é reconhecido”, chame o professor antes de continuar.

Convenção de terminal usada neste material

Ação	Linux (Mint / Ubuntu)	Windows (PowerShell)
Criar pasta	<code>mkdir app_escola</code>	<code>mkdir app_escola</code>
Entrar na pasta	<code>cd app_escola</code>	<code>cd app_escola</code>
Criar pastas aninhadas	<code>mkdir -p src/routes</code>	<code>mkdir src\routes</code>
Listar arquivos	<code>ls -la</code>	<code>dir</code>
Interromper o servidor	<code>Ctrl + C</code>	<code>Ctrl + C</code>

3. Etapa 1 — Criar o projeto e o package.json

1. Crie a pasta do projeto e entre nela.
2. Inicialize o projeto Node.js com o npm.
3. Abra a pasta no VS Code.

Terminal

```
mkdir app_escola
cd app_escola
npm init -y
code .
```

O comando `npm init -y` cria o arquivo `package.json` já preenchido com valores padrão. O `-y` aceita todas as perguntas automaticamente.

✓ CHECKPOINT

A pasta `app_escola` contém um arquivo `package.json`.

Ao abrir esse arquivo no VS Code, você vê o campo `"name": "app_escola"`.

4. Etapa 2 — Instalar as dependências

Existem dois grupos de pacotes. As dependências de produção são necessárias para a aplicação funcionar no servidor. As de desenvolvimento só existem na sua máquina, para facilitar o trabalho.

Dependências de produção

Terminal

```
npm install express express-handlebars method-override dotenv
```

Pacote	Para que serve
express	Framework que cria o servidor web e gerencia rotas e middlewares.
express-handlebars	Motor de templates: transforma arquivos <code>.handlebars</code> em HTML com dados dinâmicos.
method-override	Permite que formulários HTML enviem PUT e DELETE, que o navegador não suporta nativamente.
dotenv	Lê variáveis de ambiente de um arquivo <code>.env</code> (senhas, portas, dados de banco).

Dependências de desenvolvimento

Terminal

```
npm install --save-dev nodemon eslint @eslint/js globals
```

Pacote	Para que serve
nodemon	Reinicia o servidor sozinho toda vez que um arquivo é salvo.
eslint	Analisa o código e aponta erros e desvios de padrão.
@eslint/js	Conjunto de regras recomendadas para JavaScript.
globals	Lista de variáveis globais conhecidas (as do Node, as do navegador etc.).

 DICA

-D é a forma abreviada de --save-dev. Os dois comandos fazem exatamente a mesma coisa.
Nunca versiona a pasta node_modules. Ela é recriada com npm install a partir do package.json.

 CHECKPOINT

O package.json agora tem as seções "dependencies" e "devDependencies" preenchidas.
Surgiram a pasta node_modules e o arquivo package-lock.json.

5. Etapa 3 — Ajustar o package.json (ESM e script dev)

Duas alterações manuais são necessárias. Abra o package.json e edite.

Alteração 1: habilitar o padrão ESM

Acrescente a linha `"type": "module"`. Sem ela, o Node interpreta os arquivos no padrão antigo (CommonJS, com `require`) e a palavra `import` causa erro.

Alteração 2: criar o script de execução

Dentro de `"scripts"`, adicione `"dev": "nodemon ./server.js"`. A partir daí, o projeto inteiro é iniciado com um único comando: `npm run dev`.

package.json

```
{
  "name": "app_escola",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "type": "module",
  "scripts": {
    "dev": "nodemon ./server.js",
    "lint": "eslint ."
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "express-handlebars": "^9.0.1",
    "method-override": "^3.0.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "eslint": "^10.8.0",
    "globals": "^17.9.0",
    "nodemon": "^3.1.14"
  }
}
```

⚠ ATENÇÃO

Os números de versão do seu projeto podem ser diferentes destes, e isso é normal: o npm sempre instala a versão mais recente disponível. Não altere os números manualmente.

Cuidado com vírgulas em JSON. Não existe vírgula depois do último item de um bloco, e comentários não são permitidos.

✓ CHECKPOINT

O package.json contém `"type": "module"`.

O bloco `"scripts"` contém a linha `"dev"`.

O VS Code não mostra nenhum sublinhado vermelho no arquivo (sinal de JSON inválido).

6. Etapa 4 — Configurar o ESLint

Na raiz do projeto, crie o arquivo `eslint.config.mjs` com o conteúdo abaixo. A extensão `.mjs` garante que ele seja lido como módulo ESM.

`eslint.config.mjs`

```
import js from "@eslint/js";
import globals from "globals";
import { defineConfig } from "eslint/config";

export default defineConfig([
  {
    files: ["**/*.js", "**/*.mjs"],
    plugins: { js },
    extends: ["js/recommended"],
    languageOptions: {
      ecmaVersion: "latest",
      sourceType: "module",
      globals: globals.node,
    },
    rules: {
      "no-unused-vars": "warn",
      "no-console": "off",
    },
  },
]);
```

Para rodar a verificação manualmente:

Terminal

```
npm run lint
```



DICA

`globals.node` informa ao ESLint que variáveis como `process` e `console` existem neste ambiente. Sem isso, ele acusaria essas variáveis como não definidas.

Instale a extensão ESLint no VS Code para ver os avisos direto no editor, enquanto digita.

7. Etapa 5 — Criar a estrutura de pastas src/

Toda a lógica da aplicação fica dentro de `src/`. Na raiz permanecem apenas os arquivos de configuração e o `server.js`.

Terminal (Linux)

```
mkdir -p src/controllers src/routes src/views/layouts
```

Terminal (Windows PowerShell)

```
mkdir src\controllers
mkdir src\routes
mkdir src\views\layouts
```

Você também pode criar as pastas clicando com o botão direito na árvore de arquivos do VS Code. O resultado é o mesmo.

✓ CHECKPOINT

Dentro de `src/` existem três pastas: `controllers`, `routes` e `views`.

Dentro de `src/views/` existe a pasta `layouts`.

⚠ ATENÇÃO

Pastas vazias não aparecem em alguns sistemas de controle de versão. Elas só passam a existir de fato quando recebem o primeiro arquivo, o que faremos nas próximas etapas.

8. Etapa 6 — Criar o `src/app.js`

Este é o coração da aplicação. Crie o arquivo dentro de `src/`, e não na raiz. Por enquanto vamos escrever a versão mínima; ela será completada na Etapa 8.

src/app.js

```
import express from "express";
import methodOverride from "method-override";
import { engine } from "express-handlebars";
import { resolve } from "path";
import { fileURLToPath } from "url";

// Em ESM não existem __filename e __dirname automáticos.
// As duas linhas abaixo reconstroem esses caminhos.
const __filename = fileURLToPath(import.meta.url);
const __dirname = resolve(__filename, "..");

class App {
  constructor() {
    this.app = express();
    this.middlewares();
    this.views();
    this.routes();
  }

  middlewares() {}

  views() {}

  routes() {}
}
```

```
export default new App().app;
```

Entendendo o código

- A classe App agrupa toda a configuração em métodos com nomes claros. O constructor chama esses métodos na ordem certa: primeiro os middlewares, depois as views, por último as rotas.
- A ordem importa. O Express executa middlewares na sequência em que foram registrados; se as rotas fossem registradas antes dos middlewares, elas não teriam acesso aos dados do formulário.
- `export default new App().app` exporta a instância já configurada do Express, e não a classe. Quem importa recebe a aplicação pronta para uso.
- `__dirname` aqui aponta para a pasta `src/`, porque é onde este arquivo está. Isso será usado para localizar a pasta de views.

9. Etapa 7 — Criar o server.js

Crie o arquivo server.js na RAIZ do projeto, fora de src/.

server.js

```
import app from "./src/app.js";

const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

⚠️ ATENÇÃO

O caminho é ./src/app.js e não ./app.js. O ponto-e-barras inicial significa “a partir da pasta onde este arquivo está”, ou seja, a raiz do projeto. Como app.js está dentro de src/, é preciso atravessar essa pasta. Em ESM, a extensão .js é obrigatória nos imports de arquivos do próprio projeto. Escrever "./src/app" causa o erro ERR_MODULE_NOT_FOUND.

Por que separar server.js de app.js?

Porque são responsabilidades diferentes. O app.js descreve a aplicação; o server.js decide onde ela vai escutar. Com essa separação, é possível importar o app em testes automatizados ou publicá-lo em um serviço de hospedagem que define a porta sozinho, sem alterar uma linha da aplicação.

💡 DICA

Em projetos publicados na internet, a porta costuma vir do ambiente: const PORT = process.env.PORT || 3000. Guarde essa ideia para quando estudarmos o dotenv.

10. Etapa 8 — Configurar o Handlebars e os middlewares

Agora preencha os métodos `middlewares()` e `views()` no arquivo `src/app.js`.

src/app.js (trecho)

```
middlewares() {
  // Lê dados enviados por formulários HTML
  this.app.use(express.urlencoded({ extended: true }));
  // Lê dados enviados em formato JSON
  this.app.use(express.json());
  // Permite usar PUT e DELETE a partir de formulários
  this.app.use(methodOverride("_method"));
}

views() {
  this.app.engine(
    "handlebars",
    engine({
      defaultLayout: "main",
      layoutsDir: resolve(__dirname, "views", "layouts"),
    })
  );
  this.app.set("view engine", "handlebars");
  this.app.set("views", resolve(__dirname, "views"));
}
```

O que cada linha faz

Instrução	Efeito
<code>express.urlencoded</code>	Preenche <code>req.body</code> com os campos de um formulário HTML.
<code>express.json</code>	Preenche <code>req.body</code> quando os dados chegam em JSON (usado por APIs).
<code>methodOverride("_method")</code>	Se a URL trouxer <code>?_method=DELETE</code> , o Express trata a requisição como DELETE.
<code>app.engine</code>	Registra o Handlebars como motor capaz de processar arquivos <code>.handlebars</code> .
<code>defaultLayout: "main"</code>	Define o gabarito HTML que envolve todas as páginas.
<code>app.set("view engine")</code>	Permite chamar <code>res.render("home")</code> sem escrever a extensão do arquivo.
<code>app.set("views")</code>	Informa em qual pasta o Express deve procurar os arquivos de view.

11. Etapa 9 — Criar a primeira rota, controller e view

Vamos percorrer o caminho completo de uma requisição, do endereço digitado até o HTML exibido. São quatro arquivos, nesta ordem.

9.1 — O controller

O controller decide o que responder. Ele não sabe qual URL o acionou.

src/controllers/homeController.js

```
class HomeController {
  index(req, res) {
    return res.render("home", {
      titulo: "Sistema Escolar",
      turmas: [
        { nome: "Informática 1", alunos: 42 },
        { nome: "Informática 2", alunos: 38 },
      ],
    });
  }
}

export default new HomeController();
```

O segundo parâmetro de `res.render` é o objeto de dados que a view poderá exibir.

9.2 — A rota

A rota associa um endereço e um método HTTP a uma função do controller.

src/routes/home.js

```
import { Router } from "express";
import homeController from "../controllers/homeController.js";

const router = Router();

router.get("/", homeController.index);

export default router;
```



DICA

Repare que passamos `homeController.index` sem parênteses. Estamos entregando a função ao Express, que a executará quando alguém acessar a rota. Com parênteses, a função rodaria uma única vez, na inicialização do servidor.

9.3 — As views

O layout `main.handlebars` é a moldura HTML comum a todas as páginas. A expressão `{{{body}}}` é o ponto onde o conteúdo de cada página é encaixado.

src/views/layouts/main.handlebars

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>{{titulo}}</title>
  </head>
  <body>
    <header>
      <h1>{{titulo}}</h1>
    </header>
    <main>{{{body}}}</main>
  </body>
</html>
```

src/views/home.handlebars

```
<p>Aplicacao Express configurada com sucesso.</p>

<h2>Turmas cadastradas</h2>
<ul>
  {{#each turmas}}
    <li>{{this.nome}} - {{this.alunos}} alunos</li>
  {{/each}}
</ul>
```

⚠️ ATENÇÃO

Três chaves ({{{body}}}) inserem HTML já pronto. Duas chaves ({{titulo}}) inserem texto puro e escapam caracteres especiais, o que protege a página contra injeção de código. Use duas chaves em tudo que vier do usuário.

9.4 — Registrar a rota no app.js

Por fim, ligue o roteador à aplicação. Acrescente o import no topo do arquivo e preencha o método routes().

src/app.js (trecho)

```
// no topo do arquivo, junto aos demais imports
import homeRoutes from "./routes/home.js";

// ...

routes() {
  this.app.use("/", homeRoutes);
}
```

💡 DICA

Nesta etapa deixamos de usar this.app.use("/", (req, res) => ...). Aquela forma registra uma função para qualquer caminho e qualquer método HTTP, o que impede que outras rotas funcionem depois dela. Usar um Router deixa explícito qual método e qual endereço estão sendo tratados.

12. Etapa 10 — Executar e testar

Terminal

```
npm run dev
```

A saída esperada é:

```
[nodemon] starting `node ./server.js`  
Servidor rodando em http://localhost:3000
```

Abra o navegador em <http://localhost:3000>. Você deve ver o título, a mensagem de sucesso e a lista de turmas.

✓ CHECKPOINT

O terminal exibe a mensagem do console.log sem nenhum erro em vermelho.

O navegador exibe a página com as duas turmas listadas.

Ao alterar o texto do controller e salvar, o nodemon reinicia sozinho e a página muda ao ser atualizada.

Ctrl + C encerra o servidor.

13. Conferência final do projeto

Marque cada item somente depois de verificar de fato no seu computador.

OK	Item verificado	Como verificar
()	package.json possui "type": "module"	abrir o arquivo
()	package.json possui o script "dev"	abrir o arquivo
()	server.js está na raiz e importa ./src/app.js	abrir o arquivo
()	src/ contém app.js, routes/, controllers/ e views/	árvore do VS Code
()	src/views/layouts/main.handlebars existe	árvore do VS Code
()	npm run dev inicia sem erros	terminal
()	A página abre em http://localhost:3000	navegador
()	npm run lint não aponta erros	terminal

14. Erros comuns e como resolver

Mensagem no terminal	Causa	Solução
Cannot find module './.../app.js'	server.js está importando o caminho errado	trocar por ./src/app.js
ERR_MODULE_NOT_FOUND	faltou a extensão .js no import	escrever ./routes/home.js
Cannot use import statement outside a module	falta "type": "module"	adicionar a linha no package.json
Error: Failed to lookup view "home"	o Express não encontrou a pasta de views	conferir app.set("views", ...) e o nome do arquivo
The partial main could not be found	o layout não está no lugar esperado	conferir src/views/layouts/main.handlebars
EADDRINUSE: address already in use :::3000	a porta 3000 já está ocupada	encerrar o servidor anterior com Ctrl + C ou trocar a porta
'nodemon' não é reconhecido	nodemon não instalado ou chamado fora do npm	instalar com -D e usar npm run dev
SyntaxError: Unexpected token } in JSON	vírgula sobrando no package.json	revisar as vírgulas do arquivo
A página abre em branco	a view não foi renderizada	conferir se o controller usa res.render e o nome da view

DICA

Leia sempre a PRIMEIRA linha do erro e o caminho do arquivo citado. Na maioria dos casos, a mensagem já indica o arquivo e a linha exata do problema.

15. Atividades

1. Crie a rota GET /sobre, com um controller sobreController.js e uma view sobre.handlebars, exibindo o nome da escola, o nome do curso e o seu nome.
2. Acrescente ao array turmas do homeController mais duas turmas e confira se a listagem na página se atualiza sozinha.
3. Altere a porta do servidor para 3333 e registre no caderno qual arquivo precisou ser modificado e por quê.
4. Explique, em até cinco linhas, a diferença entre o papel do arquivo de rota e o papel do controller.
5. Provoque intencionalmente o erro Failed to lookup view. Registre o que você alterou, a mensagem completa exibida e como corrigiu.
6. Desafio: crie a pasta src/views/partials, registre partialsDir na configuração do engine e monte um menu de navegação reaproveitado pelas duas páginas.

Entrega: apresentar o projeto em execução ao professor e enviar o link do repositório no Google Sala de Aula.